

Puzzle.v is an IC design that operates with primary input I, and primary outputs success and O. The circuit operates as a lock and key type puzzle, where the correct sequence input on I will enable success and generate a specific output. I have successfully reverse engineered this circuit using a combination of methods such as: GDS design inspection, reverse compilation, a spice to verilog script, graphical inspection and formal verification methods.

I first started by checking the GDS with the tiny tapeout online viewer. From the cells/ instances menu the list of design instances can be seen. The majority of cells start with sky130_fd_sc_hd. With some research, I discovered this indicates the tech process used to synthesise the design. Sky water 130nm is a prominent open source tech process used to compile small/ non-complex ICs. The entire PDK is available on GitHub.

The pdk includes functional verilog models for all of the cells in the design. The next target was to extract some sort of netlist that would enable functional simulation. Through research I came across the tool called 'Magic'. This tool could take the PDK and GDS and produce a spice netlist, by extracting an intermediate ext file and converting to a spice file. This would then enable a quasi-decompilation by matching the spice netlist into a verilog netlist. The script `gds_extraction/extract_puzzle_gds_to_spice.tcl` shows the magic commands used for extraction.

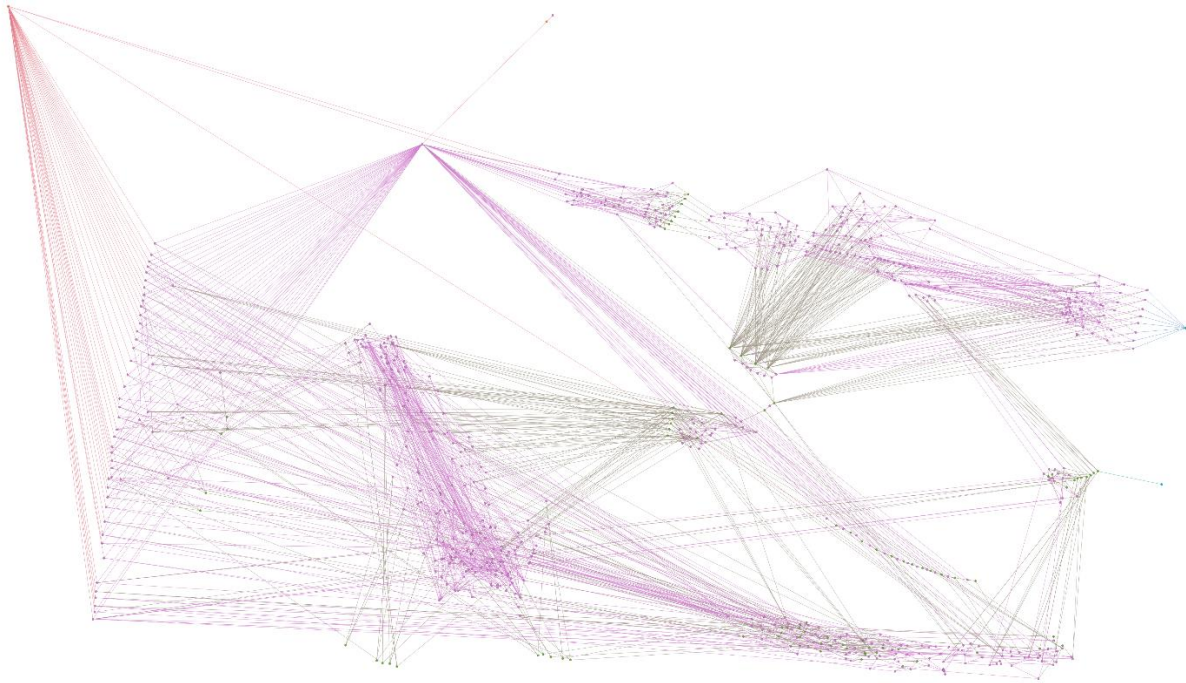
The next hurdle was to convert this spice file into a verilog netlist. This was a complex challenge and I am not aware of a popular solution. I decided to generate the netlist via a python script that would write out the cells with identical name and replicate the connection nets. This was the most challenging step in the puzzle, as I had to add a lot of features to cover edge cases and check names were legal for Verilog synthesis. I also Had to go backwards in the generation after simulation to remove power logic and cells from the verilog netlist, as they are not required for functional simulation. The sequential logic cells also did not have power ports defined which created problems for functional simulation.

To validate this netlist modelled the inputs of the example vcd file. This produced a similar result, which gave me a certain degree of confidence that this was a functional replication of the original design in the gds. I came across some hurdles when running simulation as my compiler would not work with the sequential logic definitions due to `default_nettype none`. To avoid modifying the pdk files, I created a wrapper for sequential logic to add `default_nettype wire`.

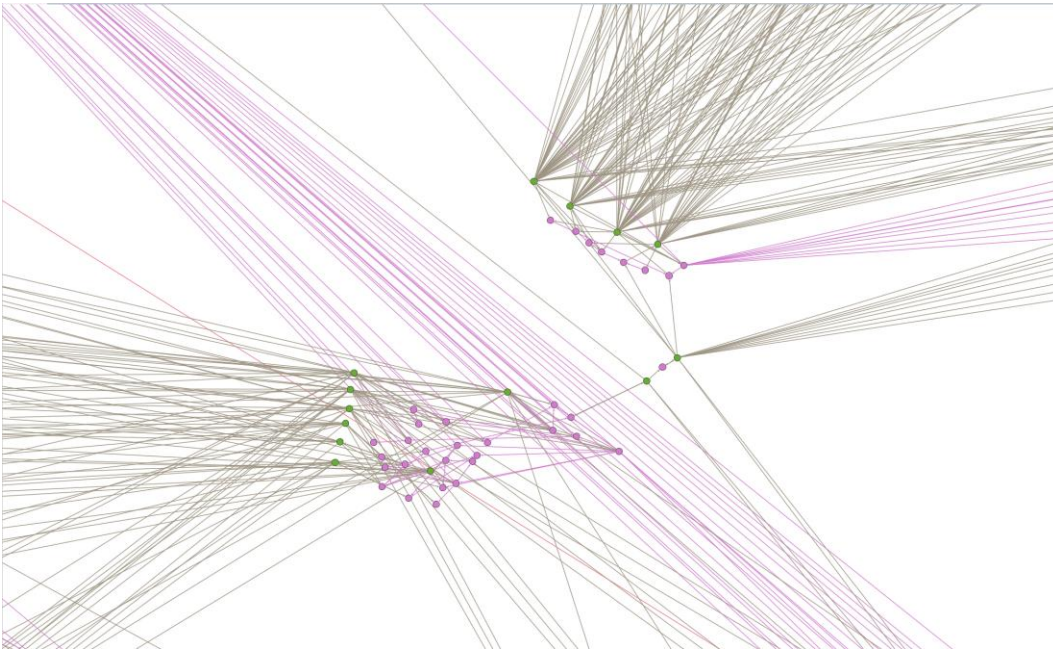
My intuition was to visually analyse the design, given that it was hinted that the structure of the design would give an insight into how it functioned. A few potential avenues were available. Digitaljs is a file format that can represent and model circuits, which can be stimulated to help understand functionality. This seemed like a suitable approach, but this format is better suited for smaller designs. I could not effectively inspect a design of this size with the tools and hardware available to me. Instead I used the graph ML format. This is a lightweight visualisation format used for large data sets, meaning I would be able to inspect a design of this scope.

The script `netlist_visualization/netlist_to_graphml.py` produces a node network with visual representations of different components. This was highly beneficial to be able to properly cluster the node network between combinational and sequential logic. The primary pattern I used cluster the design

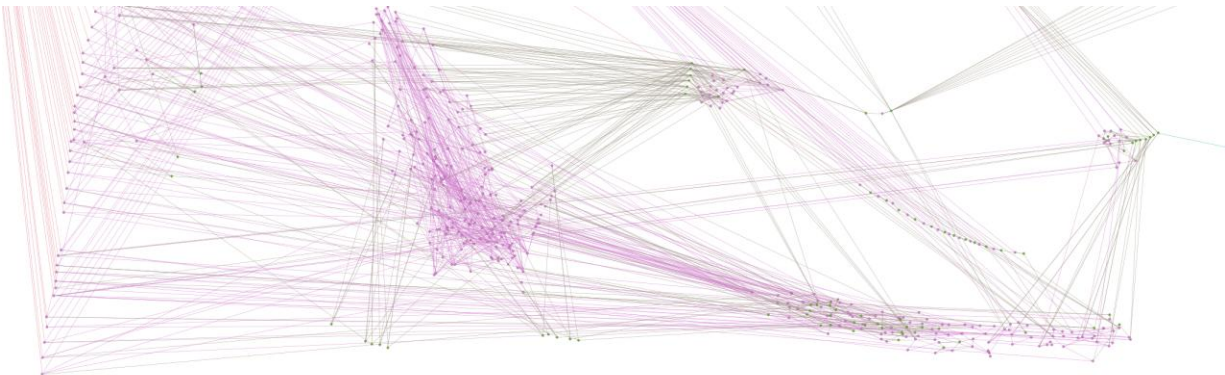
initially was to get a feed in network to every register. This was a more optimal process as registers can - by definition - only have one data input when removing the clock and reset ports. This meant I could go backwards through the network and find groups of combinational logic that fed into each register. After initial clustering I was able to discover patterns in the network that correlated to convential RTL techniques. The following Image was my final placement of nodes.



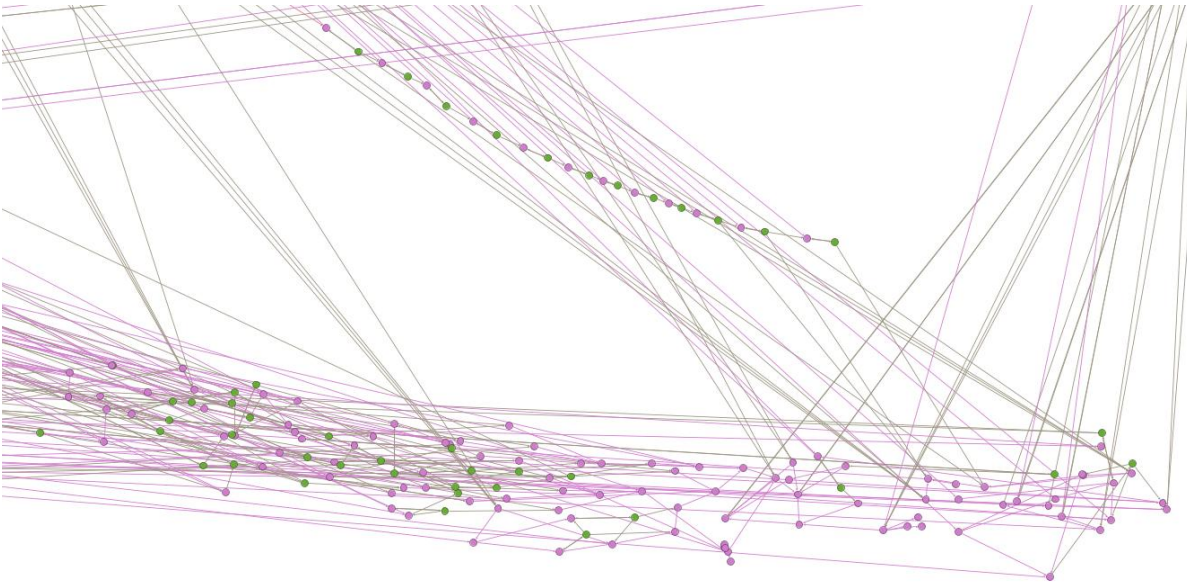
The image above shows the node network of cells for the whole design. I have included the gephi and graphml files. Combinational logic is shown in purple, sequential in green, inputs in orange, and outputs in blue. As this is a single clock and reset design the, edges for these inputs have been removed.



The first structure to highlight is what I believe to be two counters, with the top being for the output sequence and the bottom being for tracking the input stream count. The bottom counter is larger than the one above, which tracks due to the fact that the input stream is 121 bits, and the output sequence is much shorter.



The larger counter feeds into the structure shown above. It is apparent there is a larger number of combinational cells are being fed by the input stream and the counter. I speculate that in the RTL design, the password that is indexed into an array/register structure where the counter is used to track which value the input is compared to. During synthesis, this structure is then hardcoded into combinational cells with fixed 'high' or 'low' values used for comparison.



Looking in further detail at the success logic, there appears to be a higher quantity of registers in this section, that I speculate are maintaining the current state of the input stream - whether the input has been erroneous and therefore whether success can be set high. At the top of this section there does appear to be an input shift register feeding into the success logic network. The shift register has a length of 12, which matches the maximum number continuous 0 inputs in the password. It may be a way of easily tracking if there are too many consecutive 0's, however I am not confident of this assessment as there is a limited number of nets feeding from the shift register into the combinational network. I am curious if this is a feature of the design's RTL, and what it's intended use case is.



The top counter feeds into a structure resembling a FSM, with state transition side being fed by the counter and the input stream, with the output logic being fed by the current state, counter and the value off success. This FSM has different behaviour depending on success and failure.

After inspecting this network I understood that the scope of the design was further than what I would be able to logically deduce in a reasonable amount of time.

Formal verification is a modern method for verifying digital design implementations. I set out to use this approach to prove that success can be set high, and therefore discover the password. For the testing I emulated the bring-up of the design for reset and enable and constrained them as constant values. This used the same adapter cells as prior simulations.

I created an initial sanity check test to verify that success could be forced high with `vc_formal_password_extraction/ fv_run_cover_property.tcl`. Next I ran with the assertion of success = 1 with `vc_formal_password_extraction/ fv_run_confirm_assert.tcl`. The tool gave a result and exported an fsdb waveform file. After converting this to vcd in Verdi I was able to get the password of:

```
000000010101000010000000000001010101000000000000101000000100000100000010000010100001
00000001000000100000100100010100000000
```

When entering the password as success goes high, the following vectors are seen on the output.

```
28 2a 20 54 57 4f 20 53 54 41 52 53 20 2a 29
```

This can be converted to ASCII to give the following:

```
(* TWO STARS *)
```

Going back to the failure case, the following output can be seen:

```
54 52 59 20 41 47 41 49 4e
```

```
TRY AGAIN
```